Docker commands

1. Container Management Commands

These commands are used to work with Docker containers: creation, management, and lifecycle.

- `docker run`: Create and start a container from an image.
- `docker start`: Start one or more stopped containers.
- `docker stop`: Stop one or more running containers.
- `docker restart`: Restart one or more containers.
- `docker pause`: Pause all processes in one or more containers.
- `docker unpause`: Unpause all processes in one or more containers.
- `docker kill`: Kill one or more running containers.
- `docker rm`: Remove one or more containers.
- `docker ps`: List running containers.
- `docker ps -a`: List all containers (including stopped ones).
- `docker exec`: Run a command in a running container.
- `docker attach`: Attach to a running container's process.
- `docker inspect`: Return detailed information about containers, images, or volumes.
- `docker logs`: Fetch the logs from a container.
- `docker top`: Display running processes of a container.
- `docker wait`: Block until the container stops, then prints the container's exit code.

2. Image Management Commands

These commands deal with Docker images — pulling, building, tagging, etc.

- `docker build`: Build an image from a Dockerfile.
- `docker pull`: Pull an image from a registry.
- `docker push`: Push an image to a registry.
- `docker images`: List all available images locally.

- `docker rmi:` Remove one or more images.
- `docker tag:` Tag an image with a new name.
- `docker history:` Show the history of an image.
- `docker save:` Save an image to a tarball.
- `docker load:` Load an image from a tarball.

3. Network Management Commands

These commands are related to Docker networks, which manage how containers communicate.

- `docker network create:` Create a new Docker network.
- `docker network ls:` List all Docker networks.
- `docker network inspect:` View detailed information about a Docker network.
- `docker network rm:` Remove one or more networks.
- `docker network connect:` Connect a container to a network.
- `docker network disconnect:` Disconnect a container from a network.

4. Volume Management Commands

These are used to create and manage Docker volumes for persistent storage.

- `docker volume create:` Create a new volume.
- `docker volume ls`: List all Docker volumes.
- `docker volume inspect:` View detailed information about a volume.
- `docker volume rm:` Remove one or more volumes.
- `docker volume prune:` Remove all unused volumes.

5. Docker System Commands

These commands help you manage the overall Docker system, including cleaning up unused resources, viewing system info, etc.

- `docker info:` Display system-wide information about Docker.
- `docker version:` Show Docker version details.
- `docker stats:` Show real-time statistics for running containers.
- `docker events:` Get real-time events from Docker.
- `docker system df:` Show disk usage statistics (images, containers, volumes, etc.).
- `docker system prune:` Remove unused data (containers, networks, images, build cache).
- `docker system info:` Show detailed Docker system information.
- `docker login:` Log in to a Docker registry.
- `docker logout:` Log out from a Docker registry.

6. Docker Compose Commands

These commands are used for managing multi-container Docker applications (usually through a docker-compose.yml file).

- `docker-compose up:` Create and start containers defined in a docker-compose.yml file.
- `docker-compose down:` Stop and remove containers, networks, and volumes created by docker-compose up.
- `docker-compose build:` Build or rebuild services defined in a docker-compose.yml.
- `docker-compose ps:` List containers related to a docker-compose project.
- `docker-compose logs:` View logs from containers defined in a docker-compose.yml.
- `docker-compose exec:` Run a command in a running container.
- `docker-compose start:` Start services defined in a docker-compose.yml file.
- `docker-compose stop:` Stop services defined in a docker-compose.yml file.

- `docker-compose restart:` Restart services defined in a docker-compose.yml file.
- `docker-compose rm:` Remove stopped service containers.
- `docker-compose pull:` Pull images for services defined in a docker-compose.yml.
- `docker-compose push:` Push images to a registry defined in a docker-compose.yml.

7. Docker Swarm Commands

These commands are used for managing Docker Swarm clusters (for high-availability, multi-node environments).

- `docker swarm init:` Initialize a new Docker Swarm.
- `docker swarm join:` Join a node to a Docker Swarm.
- `docker swarm leave:` Make a node leave the Docker Swarm.
- `docker node ls:` List all nodes in a Docker Swarm.
- `docker node inspect:` Get detailed information about a Swarm node.
- `docker service create:` Create a new service in Docker Swarm.
- `docker service ls:` List services running in a Swarm.
- `docker service inspect:` Get detailed information about a Docker service.
- `docker service update:` Update an existing Docker service.
- `docker service scale:` Scale a Docker service up or down.
- `docker service rm:` Remove a service from the Swarm.
- `docker stack deploy:` Deploy a stack (collection of services) in a Swarm.
- `docker stack ls:` List all stacks in a Swarm.
- `docker stack rm:` Remove a stack from a Swarm.

8. Docker Build and Cache Management

These commands are specifically for managing build processes and cache.

- `docker buildx:` Extended build tool with advanced features like multi-platform builds.
- `docker build --no-cache:` Build an image without using cache.
- `docker build --pull:` Ensure the latest version of the base image is pulled when building.
- `docker build --squash:` Squash the layers of an image during the build process.

9. Docker Secrets Management

Docker secrets allow secure storage of sensitive data like passwords, tokens, etc., particularly for Docker Swarm.

- `docker secret create:` Create a secret for use in Docker Swarm.
- `docker secret ls:` List all secrets in a Swarm.
- `docker secret inspect:` Inspect a Docker secret.
- `docker secret rm:` Remove a secret from a Swarm.
- `docker secret update:` Update an existing secret in Docker Swarm.

10. Docker BuildKit Commands

Docker BuildKit is an advanced build engine that enables faster, more efficient builds with additional features such as parallel build stages and cache imports. These commands provide advanced functionality for building Docker images.

- `docker buildx build:` Build an image using BuildKit (advanced build).
- `docker buildx create:` Create a new BuildKit builder.
- `docker buildx use:` Switch between BuildKit builders.
- `docker buildx ls:` List all the BuildKit builders.
- `docker buildx bake:` A declarative way to define and build multiple images with BuildKit.

11. Docker Trusted Registry (DTR) Commands

If you're using Docker Trusted Registry (DTR), these commands help with managing and interacting with the registry.

- `docker trust:` Manage content trust for Docker images (enabled by default for DTR).
- `docker trust sign:` Sign an image for Docker Content Trust.
- `docker trust inspect:` Inspect a signed image to see its trust status.
- `docker trust revoke:` Revoke a signature from a Docker image.

12. Docker Content Trust Commands

Docker Content Trust (DCT) is a feature that ensures image authenticity and integrity using digital signatures.

- `DOCKER_CONTENT_TRUST=1:` Enable Docker Content Trust (DCT) for signing and verifying images.
- `DOCKER_CONTENT_TRUST=0:` Disable DCT (not recommended for production).

13. Docker Registry Commands

If you're working with Docker's private registry (Docker Hub or custom registry), these commands are useful for interacting with it.

- `docker login :` Log in to a custom Docker registry.
- `docker logout :` Log out from a custom Docker registry.
- `docker search :` Search for images on Docker Hub or a configured registry.
- `docker pull :` Pull an image from the registry.
- `docker push :` Push an image to the registry.

14. Docker Security Commands

These commands are used to manage security settings, vulnerabilities, and user access in Docker environments.

- `docker scan:` Perform security scanning on an image to detect vulnerabilities (requires Docker Scan or integration with a tool like Snyk).
- `docker info --security:` Display security-related information about the Docker system.
- `docker history --no-trunc:` Show the full, untruncated history of an image, useful for auditing security.
- `docker inspect --format '{{.Config.Entrypoint}}':` Extract the entry point of an image for security analysis (useful for determining the attack surface).
- `docker update --security-opt:` Apply additional security options to a container, such as user namespaces or seccomp profiles.

15. Docker Logs and Monitoring Commands

For monitoring containers, fetching logs, and interacting with running containers.

- `docker logs -f :` Tail logs from a container in real-time.
- `docker logs --timestamps :` View logs with timestamps for better log management.
- `docker stats --no-stream:` Get one-time resource usage statistics.
- `docker stats --all:` Display stats for all containers, including those not currently running.
- `docker events --filter event=start:` Listen for specific events, like container start or stop.
- `docker top :` Display processes running inside a container.

16. Docker Compose Override Commands

docker-compose.override.yml is used to override settings from docker-compose.yml. These commands are important for working with multiple configuration files.

- `docker-compose -f up:` Use a specific docker-compose.yml file to spin up the services.
- `docker-compose -f -f up:` Use an override file along with the base file for Docker Compose.
- `docker-compose -f config:` View the effective configuration of the Docker Compose files after merging.

17. Docker System Commands for Cleanup

Commands related to cleaning up unused resources, freeing up space, and ensuring your Docker environment remains tidy.

- `docker system prune -a:` Prune everything, including unused images, containers, networks, and volumes.
- `docker image prune:` Remove unused images (not associated with any container).
- `docker container prune:` Remove stopped containers.
- `docker volume prune:` Remove unused volumes.
- `docker network prune:` Remove unused networks.
- `docker builder prune:` Remove build cache.

18. Docker Events and Monitoring

These commands are useful for getting real-time updates and monitoring changes in your Docker environment.

- `docker events:` Display real-time events from Docker, such as container stops, starts, or network changes.

- `docker stats --format:` Display statistics in a custom format, such as JSON or CSV, useful for monitoring and automation.
- `docker events --filter event=create:` Filter events to show only creation events (e.g., container or image creation).

Jayant_Sonone